

AI Journalist Architecture Overview

Current-state architecture of the AI Journalist application, based on the live codebase. This report covers the frontend, backend, LangGraph orchestration, all agents, benchmark corpora, storage layers, external services, and deployment topology across local Docker and Fly.io.

GENERATED 2026-04-29 Workspace date in Asia/Dubai	FRONTEND STACK Next.js 15 React 19, React Query, Axios	BACKEND STACK FastAPI SQLAlchemy async, LangGraph, Alembic	HOSTED DATABASE Neon Postgres Fly deployment path
--	---	---	---

1. Executive Summary

AI Journalist is a full-stack documentary-production application. It starts from a story topic, runs a multi-agent research and writing pipeline, stores intermediate artifacts in Postgres JSON fields, uploads final scripts to S3-compatible storage, and exposes both editorial and benchmark scoring back to the user.

Core architectural idea: the app treats each story as a persisted workflow instance. The FastAPI backend launches long-running background jobs, streams status updates to the frontend, and writes every major artifact back into the `stories` table so the UI can resume work, branch revisions, and run follow-up research.

PRIMARY USER PATH New Story -> research pipeline -> final script -> evaluation -> manual rewrite or targeted follow-up research.	PRIMARY ADMIN PATH Admin Console -> user management -> benchmarking console -> corpus health, reference docs, rebuild orchestration.
LOCAL ENVIRONMENT Docker Compose runs frontend, backend, local Postgres, and LocalStack. Neon is not required locally.	FLY ENVIRONMENT Frontend and backend deploy as separate Fly apps. Hosted Postgres is expected via <code>DATABASE_URL</code>, designed for Neon.

2. System Schematic

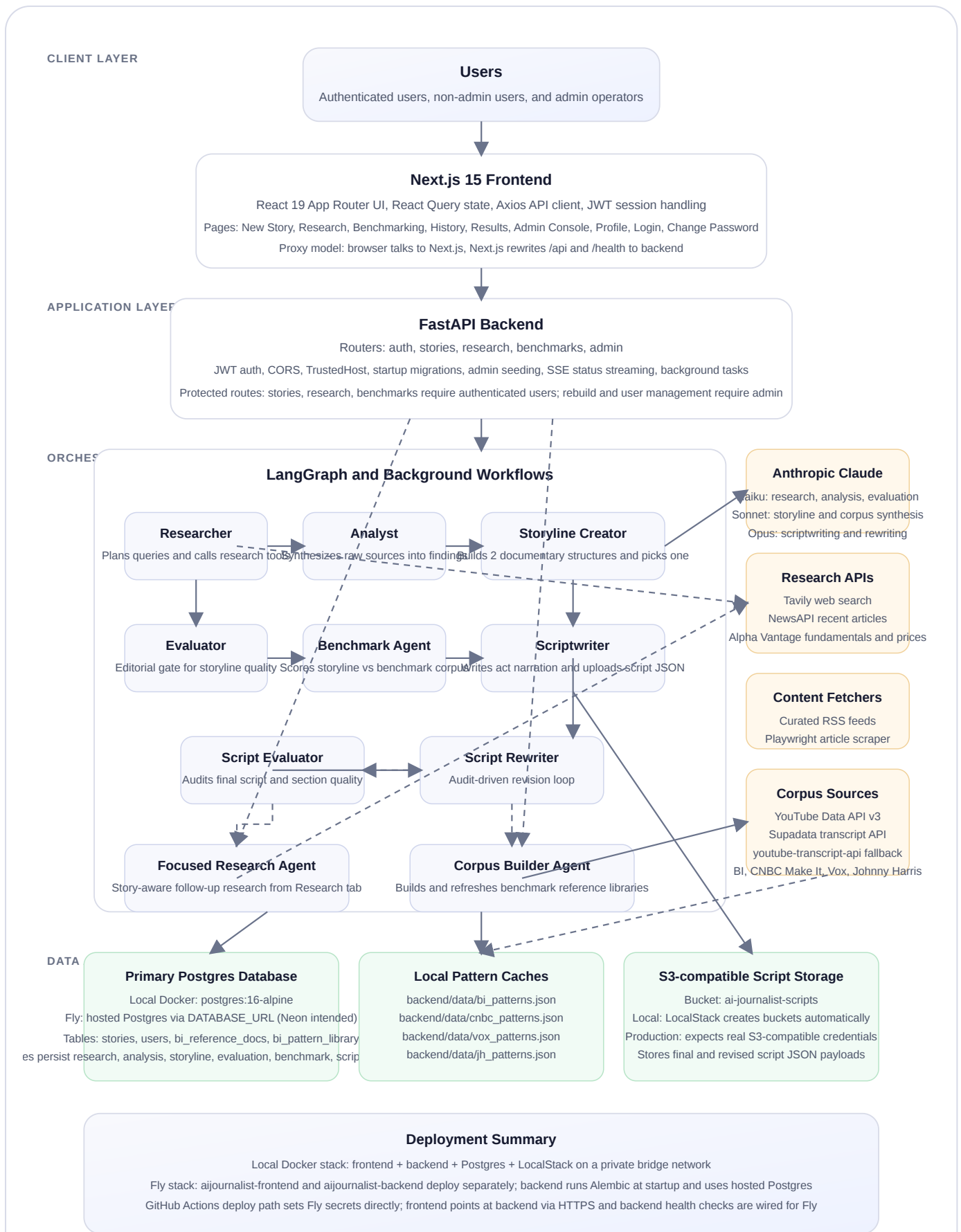


Figure 1. High-level architecture. The main LangGraph pipeline handles initial story generation, while focused research, corpus building, and rewrite workflows run as separate background tasks against the same persisted story model.

3. Product Surfaces

3.1 User-facing pages

- `/` New Story workspace: topic entry, tone selection, target duration, live pipeline progress.
- `/research` Story-specific research workspace with focused follow-up research and source browser.
- `/benchmarking` Script-scoring workspace with benchmark metrics and heuristic-driven improvement actions.
- `/history` Story archive with filters, download, and delete actions.
- `/results/[id]` Final script view, evaluation tabs, and rewrite entry point.
- `/results/[id]/evaluation` Detailed editorial quality breakdown.
- `/profile` and `/change-password` account lifecycle screens.

3.2 Admin-only pages

- `/admin` Admin Console with sub-tabs for User Management and Benchmarking Console.
- User Management calls admin-only routes to create and delete users.
- Benchmarking Console shows corpus health, reference inventory, stale status, and rebuild controls.
- Admins still use the same standard `/benchmarking` workspace as regular users for script benchmarking.

The shared frontend shell is built in Next.js App Router. `AuthGuard` blocks access to all pages except login and registration, and forces non-admin users through a password-change step when their account has `must_change_password` set.

4. Frontend Architecture

Area	Implementation	Role in the system
Framework	<code>Next.js 15.1.0</code> + <code>React 19</code>	App Router UI with server proxy and standalone production build.
State and fetching	<code>@tanstack/react-query</code>	Caches story lists, benchmark status, and polling states for long-running backend jobs.
HTTP client	<code>frontend/lib/api.ts</code> with <code>Axios</code>	Typed client for auth, stories, research, benchmarks, and admin APIs.
Auth persistence	<code>frontend/lib/auth.ts</code>	Stores JWT and cached user identity in <code>localStorage</code> or <code>sessionStorage</code> .
Proxy pattern	<code>frontend/next.config.ts</code>	Rewrites <code>/api/*</code> and <code>/health</code> to the backend so the browser talks to the frontend origin only.
Navigation shell	<code>Sidebar</code> + <code>SidebarWrapper</code>	Exposes workspace navigation and admin navigation based on stored user role.
Status updates	React Query polling + SSE stream	Most pages poll; story-generation pages also subscribe to <code>/events</code> for near-live status refresh.

Important UI pattern: the frontend is intentionally thin. It does not perform generation logic locally. It orchestrates backend jobs, resumes them after navigation, and surfaces artifacts already stored in the backend database.

5. Backend Architecture

5.1 Application bootstrap

- `backend/api/main.py` creates the FastAPI app, mounts routers, enables CORS, and applies trusted-host controls.
- On startup, the backend runs Alembic migrations against non-SQLite databases and seeds the configured admin user.
- `/health` is left reachable for Fly health checks through a selective trusted-host bypass.

5.2 Router layout

Router	Prefix	Purpose
Auth	<code>/api/v1/auth</code>	Login, current user, dismiss password-change prompt, change password.
Stories	<code>/api/v1/stories</code>	Create stories, read status, stream events, read scripts, focused research, chat, regenerate, rewrite, delete.
Research	<code>/api/v1/research</code>	Direct tool-level access to Tavily, NewsAPI, RSS, and Alpha Vantage.
Benchmarks	<code>/api/v1/benchmarks</code>	Corpus status, library catalog, reference docs, and rebuild trigger.
Admin	<code>/api/v1/admin</code>	List, create, and delete users.

5.3 Auth and input controls

- JWTs are minted on login and verified through `get_current_user`.
- Stories, research, and benchmarks routes require an authenticated user at the router level.
- Admin-only actions layer `get_admin_user` on top of general auth.
- User topic and chat inputs pass through security validators that reject code snippets and architecture-probing prompts before any LLM call.

6. LangGraph Pipeline and Agent Responsibilities

The main compiled graph lives in `backend/graph/journalist_graph.py`. Each story starts at `researcher` and carries a shared `JournalistState` object through the pipeline.

Main graph order: `researcher -> analyst -> storyline_creator -> evaluator` (with the benchmarker running in parallel inside the evaluator node) `-> scriptwriter -> script_evaluator`, with an optional loop into `script_rewriter`.

Agent	Model / strategy	Primary responsibility	Writes back into state
ResearcherAgent	Claude Haiku + research tools	Classifies topic, decomposes queries, calls routed sources in parallel, scrapes top URLs.	<code>research_package</code>
AnalystAgent	Claude Haiku structured output	Turns raw sources into executive summary, key findings, quotes, controversies, and data gaps.	<code>analysis_result</code>

StorylineCreatorAgent	Claude Sonnet structured output	Creates two 10-15 minute documentary structures and selects the stronger proposal.	<code>storyline_proposals</code> , <code>selected_storyline</code>
EvaluatorAgent	Claude Haiku structured output	Scores the selected storyline across editorial criteria and decides if it is ready for scripting.	<code>evaluation_report</code> , routing flags
BenchmarkAgent	Claude Haiku structured output + benchmark library	Scores the storyline against the benchmark corpus and produces benchmark-grade explanations.	<code>benchmark_report</code>
ScriptwriterAgent	Claude Opus structured output	Writes each act in parallel, enforces source IDs, computes word count, and uploads final script JSON.	<code>final_script</code> , <code>script_s3_key</code>
ScriptEvaluatorAgent	Claude Haiku structured output	Audits the finished script itself, including section-by-section notes and benchmark comparison.	<code>script_audit_report</code>
ScriptRewriterAgent	Claude Opus structured output	Rewrites weak sections against audit feedback and source-linked facts.	<code>final_script</code> , revision cycle
FocusedResearchAgent	Claude Haiku + research tools	Runs user-requested follow-up research for an existing story from the Research tab.	Background merge into <code>research_data</code>
CorpusBuilderAgent	Claude Haiku + Claude Sonnet + YouTube tools	Builds benchmark corpora from reference channels and synthesizes per-library pattern caches.	<code>bi_reference_docs</code> , <code>bi_pattern_library</code> , JSON caches

7. Story Lifecycle and Revision Flows

7.1 Initial story generation

1. User submits topic, tone, duration, and default audience from the New Story page.
2. The backend creates a `stories` row with status `pending`.
3. A FastAPI background task runs the compiled LangGraph pipeline.
4. Each completed node updates the story status, and the final pipeline pass persists research, analysis, storyline, evaluation, benchmark, script audit, and script JSON back to the database.
5. The frontend watches status through polling and the story events stream.

7.2 Focused research and regeneration

1. The Research page starts a background focused-research job for a selected story.
2. FocusedResearchAgent uses the existing story, audit, benchmark, and source context to decide which sources to query.
3. Returned sources are deduplicated and merged into `story.research_data`; the latest focused run is stored separately for UI recovery.
4. The user can then trigger script regeneration, which re-runs analysis, storyline creation, evaluation, benchmark scoring, scriptwriting, and script audit from the updated research set.

7.3 Rewrite and recommendation flows

1. Audit-driven rewrite clones the story into a new revision row and runs ScriptEvaluator -> ScriptRewriter -> ScriptEvaluator.
2. Implement Recommendations also clones the story, creates a synthetic audit from selected recommendations, then reuses the rewrite path.
3. Revision lineage is tracked via `parent_story_id` and `revision`.
4. Previous script snapshots are stored in `script_versions` for comparison in the UI.

8. Data Model and Storage Layers

8.1 Primary relational data

Table	Purpose	Notable fields
stories	One persisted story workflow plus all major artifacts.	topic, status, research_data, analysis_data, storyline_data, evaluation_data, benchmark_data, script_data, script_audit_data, script_versions, parent_story_id, revision
users	Authentication and admin-role tracking.	email, hashed_password, is_admin, must_change_password
bi_reference_docs	Benchmark reference videos across supported libraries.	library_key, youtube_id, title, view_count, like_count, transcript, extracted_structure
bi_pattern_library	Versioned synthesized benchmark patterns per library.	library_key, version, doc_count, patterns

8.2 Non-relational storage

Store	Format	Used for
Pattern caches in backend/data	JSON files	Fast local loading of benchmark libraries and DB fallback repair.
S3-compatible script storage	JSON object per script	Durable storage of generated scripts and revisions outside the database row.
Frontend auth storage	localStorage / sessionStorage	JWT token and cached user identity.

Operational nuance: local development uses LocalStack for S3 bucket emulation, but Fly deployment expects valid S3-compatible credentials if script-object uploads need to work outside the local environment.

9. External Services and Integrations

Integration	Where used	Purpose
Anthropic Claude	All agent classes	Reasoning and structured generation across research, analysis, storyline creation, evaluation, benchmarking, scriptwriting, and rewriting.
Tavily	WebSearchTool	Open-web background research and query-driven search results.
NewsAPI	NewsAPITool	Recent news coverage and top-headline retrieval.

Alpha Vantage	<code>FinancialDataTool</code>	Company overviews, daily prices, and ticker search for financially oriented stories.
RSS feeds + Google News RSS	<code>RSSParserTool</code>	Curated editorial coverage and trade-press monitoring.
Playwright + BeautifulSoup	<code>WebScraperTool</code>	JavaScript-capable article extraction for top web results.
YouTube Data API v3	<code>YouTubeFetcher</code>	Channel video discovery for benchmark corpora and YouTube search in story chat.
Supadata transcript API	<code>YouTubeFetcher</code>	Primary transcript source during corpus builds.
youtube-transcript-api	<code>YouTubeFetcher</code>	Fallback transcript extraction when Supadata is unavailable or exhausted.
Postgres	SQLAlchemy async engine	Primary durable application database.
Neon Postgres	Fly deployment design	Hosted Postgres provider through <code>DATABASE_URL</code> .
LocalStack	<code>docker-compose.yml</code>	Local S3 emulation for script uploads and asset buckets.

10. Benchmarking Subsystem

Benchmarking 2.0 is a dedicated subsystem rather than a small add-on. It has its own reference corpus tables, per-library caches, admin health reporting, and a separate corpus-builder agent.

SUPPORTED LIBRARIES

Business Insider, CNBC Make It, Vox, and Johnny Harris.

PRODUCTION SCORING VIEW

A combined benchmark corpus synthesized from all active component libraries.

REFRESH POLICY

Admin-triggered refresh can rotate the freshest 25% of each healthy corpus.

10.1 Benchmarking flow

1. CorpusBuilderAgent fetches documentary-length videos from a configured channel ID or handle.
2. It retrieves transcripts, extracts `DocStructure` with Claude Haiku, and persists reference docs.
3. Claude Sonnet synthesizes cross-corpus patterns into a versioned `BIPatternLibrary`.
4. The BenchmarkAgent uses the active combined library during story evaluation.
5. The frontend exposes both user-facing benchmark scoring and admin-facing benchmark health operations.

10.2 Reliability guardrails

- Library health tracks freshness, readiness, cache presence, and minimum document counts.
- If a corpus is missing or below the minimum size, admin status recommends a rebuild.
- Missing transcripts or failed structure extraction are surfaced in rebuild error messages.

11. Deployment Topology

11.1 Local Docker topology

- `frontend` : Next.js app on port 3000.
- `backend` : FastAPI app on an internal port 8000.
- `db` : local Postgres 16 container.
- `localstack` : S3 emulation with bootstrap bucket creation.

The backend service in `docker-compose.yml` explicitly overrides `DATABASE_URL` to point at the internal `db` container, which means local Docker runs without Neon.

11.2 Fly.io topology

- `aijournalist-backend` : Python + Playwright image, health-checked at `/health` , sized for Playwright memory needs.
- `aijournalist-frontend` : standalone Next.js runtime that proxies API traffic to the backend Fly app.
- GitHub Actions deployment sets Fly secrets directly, including `DATABASE_URL` .
- Hosted Postgres is expected through Fly secrets; the intended provider is Neon.

11.3 Environment split

Environment	Database target	Object storage target
Local Docker	Bundled Postgres container	LocalStack buckets
Local non-Docker	Whatever <code>.env</code> points to	S3-compatible endpoint defined in <code>.env</code>
Fly	Hosted Postgres from <code>DATABASE_URL</code> (designed for Neon)	Real S3-compatible service if production credentials are supplied

12. Operational Controls, Observability, and Failure Modes

- Story runs are resilient to navigation because long-running work is launched as backend background tasks rather than kept inside the browser.
- The story events endpoint streams periodic snapshots until a story reaches `completed` or `failed` .
- Every graph node is wrapped with error logging, and terminal failures are written into `stories.error_message` .
- Benchmark rebuild state is tracked in an in-process build-state object so the admin UI can show running, finished, and failed rebuilds.
- Migrations run automatically on startup for non-SQLite environments, reducing drift between deployed code and database schema.
- Trusted-host middleware includes a Fly-specific bypass for health endpoints because Fly health probes may send raw machine IPv6 addresses as the Host header.

Current architectural dependency to watch: story quality depends on multiple external providers remaining healthy at once: Anthropic, Tavily, NewsAPI, Alpha Vantage, and the benchmark corpus sources. The product architecture is designed to degrade gracefully in some places, but missing credentials or upstream failures still directly affect functionality.

13. Source Files Reviewed

Backend core

- backend/api/main.py
- backend/api/routes/auth.py
- backend/api/routes/admin.py
- backend/api/routes/research.py
- backend/api/routes/benchmarks.py
- backend/api/routes/stories.py
- backend/api/security.py
- backend/db/database.py
- backend/config.py

Graph, agents, and services

- backend/graph/journalist_graph.py
- backend/graph/state.py
- backend/agents/researcher.py
- backend/agents/analyst.py
- backend/agents/storyline_creator.py
- backend/agents/evaluator.py
- backend/agents/benchmark.py
- backend/agents/scriptwriter.py
- backend/agents/script_evaluator.py
- backend/agents/script_rewriter.py
- backend/agents/focused_researcher.py
- backend/agents/corpus_builder.py
- backend/services/benchmarking.py
- backend/services/script_storage.py

Models and tools

- backend/models/story.py
- backend/models/research.py
- backend/models/benchmark.py
- backend/models/user.py
- backend/tools/web_search.py
- backend/tools/news_api.py
- backend/tools/financial_data.py
- backend/tools/rss_parser.py
- backend/tools/web_scraper.py
- backend/tools/youtube_fetcher.py

Frontend and deployment

- frontend/package.json
- frontend/next.config.ts
- frontend/app/layout.tsx
- frontend/app/page.tsx
- frontend/app/research/page.tsx
- frontend/app/benchmarking/page.tsx
- frontend/app/admin/page.tsx
- frontend/components/Sidebar.tsx
- frontend/components/AuthGuard.tsx
- frontend/components/benchmarking/AdminBenchmarkView.tsx
- frontend/lib/api.ts
- frontend/lib/auth.ts
- docker-compose.yml
- fly.backend.toml
- fly.frontend.toml
- .github/workflows/fly-deploy.yml